🗄️ Vector Stores in LangChain

1. Why Do We Need Vector Stores?

🔹 **Problem Setup: IMDb-like Movie Catalog**

- You build a website listing all movies (like IMDb).
- Database stores: **movie_id, name, director, actors, genre, release_date, outcome (hit/flop), etc.**
- Data can be fetched via APIs or web scraping.
- Backend (Python) serves data → frontend displays it.

👉 Works fine for **basic search**.

🔹 **Adding a Recommender System (First Attempt)**

- Goal: If user views *Spiderman*, also recommend similar movies (*Iron Man, Captain America*).
- Approach: **Keyword Matching** (compare metadata).
  - Same director, actors, genre, release period → movies considered similar.

Problem:

- Sometimes **bad recommendations**:
  - *My Name is Khan* → suggested *Kabhi Alvida Na Kehna* (same director, actor, genre but different storyline).
- Sometimes **misses good similarities**:
  - *Taare Zameen Par* & *A Beautiful Mind* (both about struggles + brilliance) but keywords differ → system fails.

👉 Keyword matching is **too shallow**.

🔹 **Better Approach: Compare Story/Plot**

- Instead of metadata, compare **plot (storyline)**.
- Two movies with similar plots → good recommendations.

Challenge:

- Each plot = long text (2000–3000 words).
- Need a way to compare **text meaning** (semantics).
- Comparing raw text meaning is a **hard NLP task**.

🔹 **Solution: Embeddings**

- Use **deep learning embeddings**:
  - Convert text → vector of numbers.
  - Captures **semantic meaning** of text.
  - Example: 512-dimensional or 768-dimensional vectors.

Steps:

1. Extract plot text for each movie.
2. Generate embedding vectors using an embedding model.
3. Store embeddings.
4. For a given query/movie, compare embeddings with others.
5. Use **cosine similarity** (or distance) to find closest ones.

👉 Similar vectors → similar meaning.

---

2. Challenges Without Vector Stores

When building such a system:

1. **Embedding Generation**

- Need embeddings for millions of movies.

2. **Storage**
   - Normal relational DBs (MySQL, Oracle) can't store embeddings efficiently.
   - They don't support similarity search.
3. **Semantic Search Efficiency**
   - Query vector vs. millions of vectors = very expensive.
   - Need **smart indexing** (ANN – Approximate Nearest Neighbors).

👉 Vector Stores solve all 3 problems.

---

3. What is a Vector Store?
Definition:
A **system designed to store and retrieve data represented as numerical vectors (embeddings)**.

---

4. Core Features of Vector Stores

1. **Storage**
   - Store embeddings + metadata (movie_id, name, genre).
   - Options:
     - **In-memory** (RAM) → fast, temporary.
     - **On-disk** (persistent storage, DBs) → durable, scalable.
2. **Similarity Search**
   - Given a query vector, retrieve most similar vectors.
   - Enables **semantic search & recommendations**.
3. **Indexing**
   - Optimizes similarity search on high-dimensional vectors.
   - Example:
     - Cluster 1M vectors into 10 clusters.
     - Compute centroid for each.
     - Query compared only with nearest cluster.
     - Reduces computations from 1M → ~100k.
   - Popular methods: **Clustering, ANN (Approximate Nearest Neighbor)**.
4. **CRUD Operations**
   - Add, retrieve, update, delete vectors.
   - Similar to traditional databases.

---

5. Vector Stores vs Vector Databases

- **Vector Store:**
  - Lightweight system.
  - Focus on storage + similarity search.
  - Example: **FAISS (Facebook AI Similarity Search)**.
  - Best for prototyping, small-scale apps.
- **Vector Database:**
  - Vector Store + database-like features:
    - Distributed architecture
    - Backup & restore
    - ACID transactions
    - Concurrency control
    - Authentication/Authorization
  - Examples: **Pinecone, Milvus, Weaviate, Qdrant**.
  - Best for production, large-scale apps.

👉 **All Vector Databases are Vector Stores, but not all Vector Stores are Vector Databases.**

---

6. LangChain and Vector Stores

- LangChain provides **wrappers for multiple vector stores**:
  - FAISS
  - Pinecone
  - Chroma
  - Weaviate
  - Qdrant
- All share **common methods** → easy to switch.

Common Methods:

- from_documents() / from_texts() → create vector store.
- add_documents() → add new vectors.
- similarity_search() → semantic search.
- Metadata filtering.

👉 If you switch from FAISS → Pinecone → Chroma, **code remains same**.

---

7. ChromaDB Example with LangChain

🔹 Step 1: Install & Import

python Copy code

```
!pip install langchain chromadb openai
```

python Copy code

```
from langchain_openai import OpenAIEmbeddings
from langchain.vectorstores import Chroma
from langchain.schema import Document
```

🔹 Step 2: Create Documents

```
docs = [
    Document(page_content="Virat Kohli - batsman", metadata={"team": "RCB"}),
    Document(page_content="Rohit Sharma - batsman", metadata={"team": "MI"}),
    Document(page_content="MS Dhoni - wicketkeeper", metadata={"team": "CSK"}),
    Document(page_content="Jasprit Bumrah - bowler", metadata={"team": "MI"}),
    Document(page_content="Ravindra Jadeja - allrounder", metadata={"team": "CSK"})
]
```

🔹 Step 3: Create Vector Store

```
embeddings = OpenAIEmbeddings()
vectorstore = Chroma(
    collection_name="sample",
    embedding_function=embeddings,
    persist_directory="my_chroma_db"
)
```

🔹 **Step 4: Add Documents**

vectorstore.add_documents(docs)

🔹 **Step 5: Query (Semantic Search)**

```
results = vectorstore.similarity_search("Who is a bowler?", k=2)
for r in results:
    print(r.page_content, r.metadata)
```

✅ Output: Jasprit Bumrah, Ravindra Jadeja

🔹 **Step 6: Similarity Search with Score**

results = vectorstore.similarity_search_with_score("Who is a bowler?", k=2)

🔹 **Step 7: Metadata Filtering**

```
results = vectorstore.similarity_search(
    "",  # empty query
    k=5,
    filter={"team": "CSK"}
)
```

✅ Output: Dhoni, Jadeja

🔹 **Step 8: Update Document**

vectorstore.update_document(
  ids=["kohli_id"],
  documents=[Document(page_content="Former RCB captain, aggressive leader", metadata={"team": "RCB"})]
)

🔹 **Step 9: Delete Document**

vectorstore.delete(ids=["kohli_id"])

---

8. Interview Q&A

Q1. Why can't we use relational DBs (MySQL, Oracle) for embeddings?

👉 They can't efficiently compute similarity between high-dimensional vectors.

Q2. How do vector stores make semantic search fast?

👉 Using **indexing techniques** (e.g., clustering, ANN).

Q3. Difference between Vector Store & Vector Database?

👉 Vector DB = Vector Store + database features (transactions, scaling, authentication).

Q4. Which vector store is best for prototyping?

👉 FAISS or Chroma (lightweight, local).

Q5. Which vector DBs are used in production?

👉 Pinecone, Weaviate, Milvus, Qdrant.

Q6. Why does LangChain support multiple vector stores?

👉 Flexibility: switch providers without changing code.

---

9. Real-World Applications

- **RAG Applications** (QA with PDFs, research papers).
- **Movie/Book Recommendation Systems.**
- **Semantic Search Engines.**
- **Customer Support Chatbots.**
- **Enterprise Knowledge Base Search.**
- **Image/Video similarity search.**

✅ **Summary:**

- Vector Stores solve **storage, retrieval, and semantic search** for embeddings.
- They are essential for **RAG pipelines**.
- LangChain provides a unified interface for multiple vector stores.
- ChromaDB is best for **local development**, Pinecone/Weaviate for **production**.
